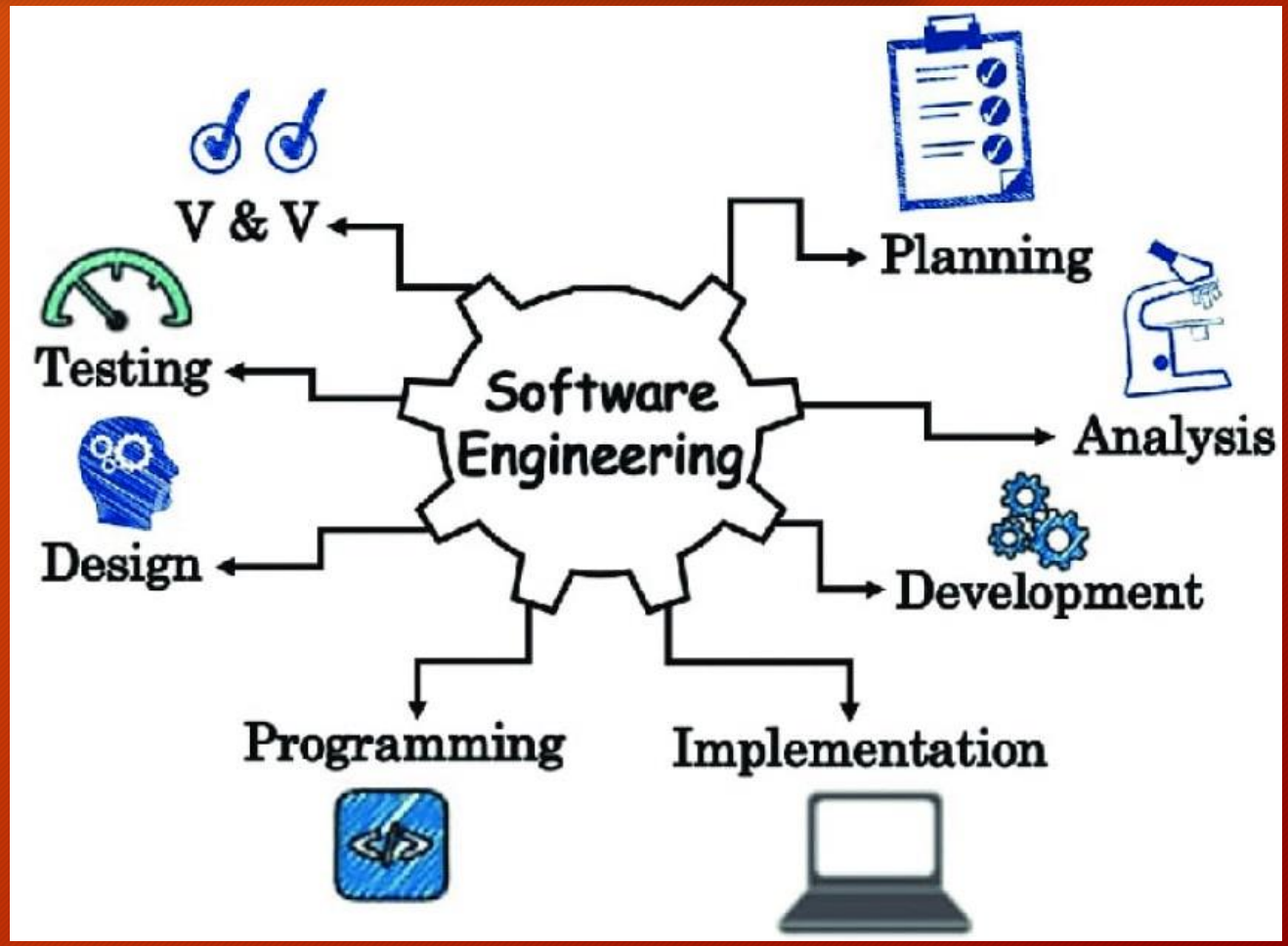
Introduction to Software Engineering

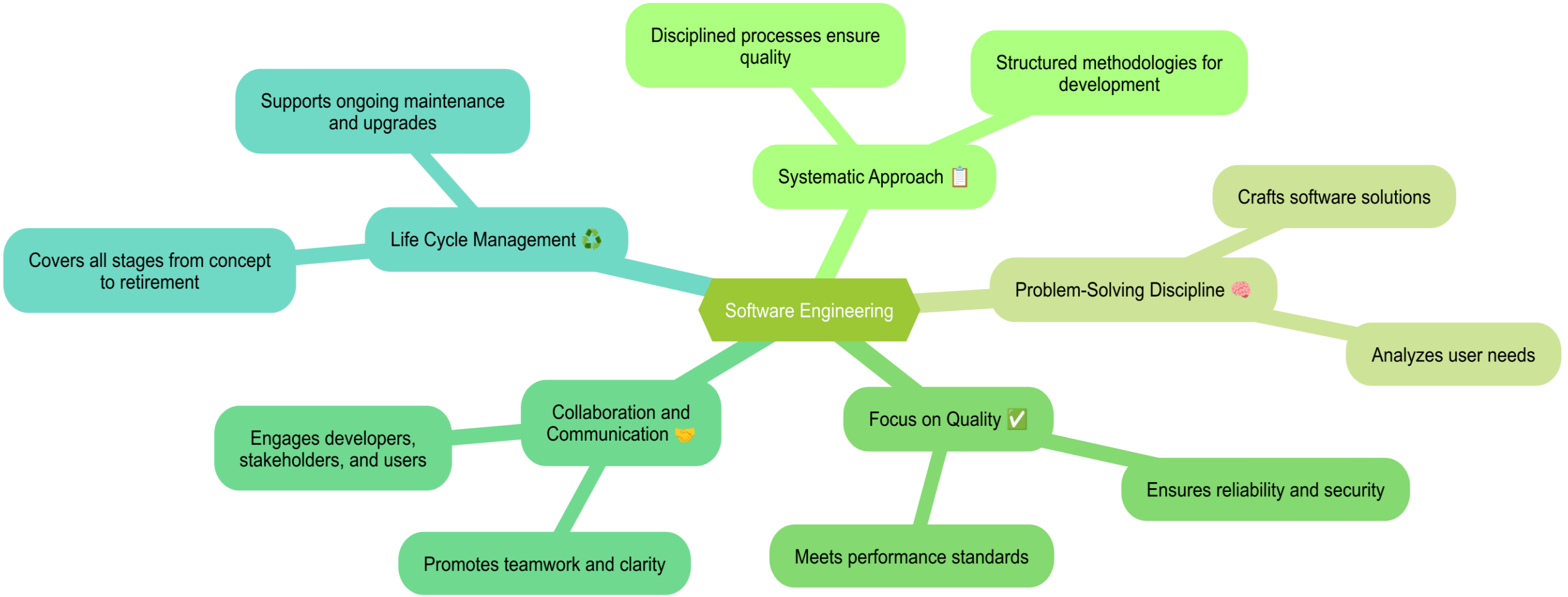
Dr. Deepanjal Shrestha
Associate Professor, School of Engineering
25/11/2024

Software Engineering

- The application of a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software (IEEE)
- Software engineering is the branch of computer science that deals with the design, development, testing, and maintenance of software applications. (Wikipedia)



Software Engineering

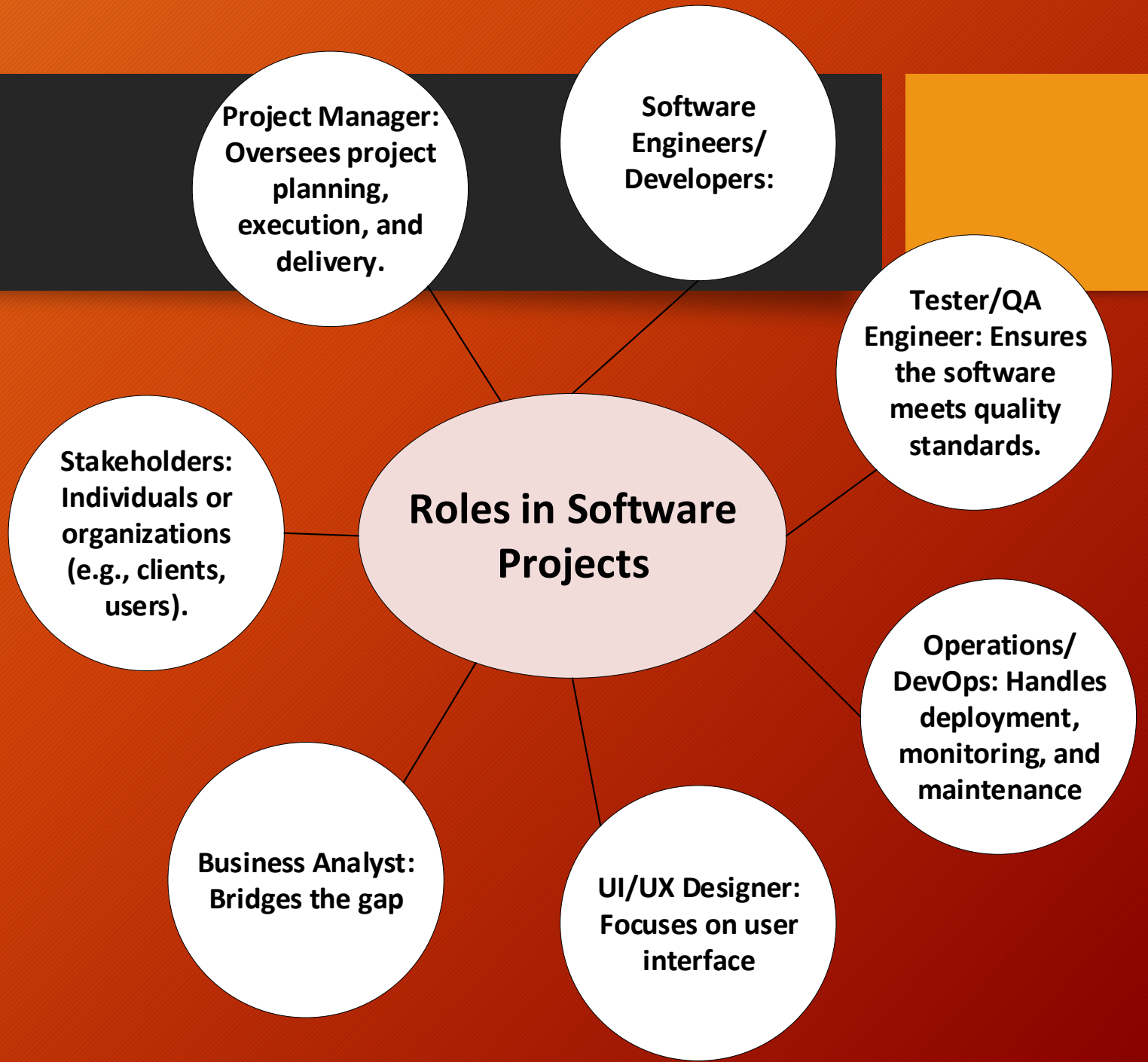


Evolution of Software Engineering



Participants and Roles

Who's role do you think is more important in Software Project?? Why?



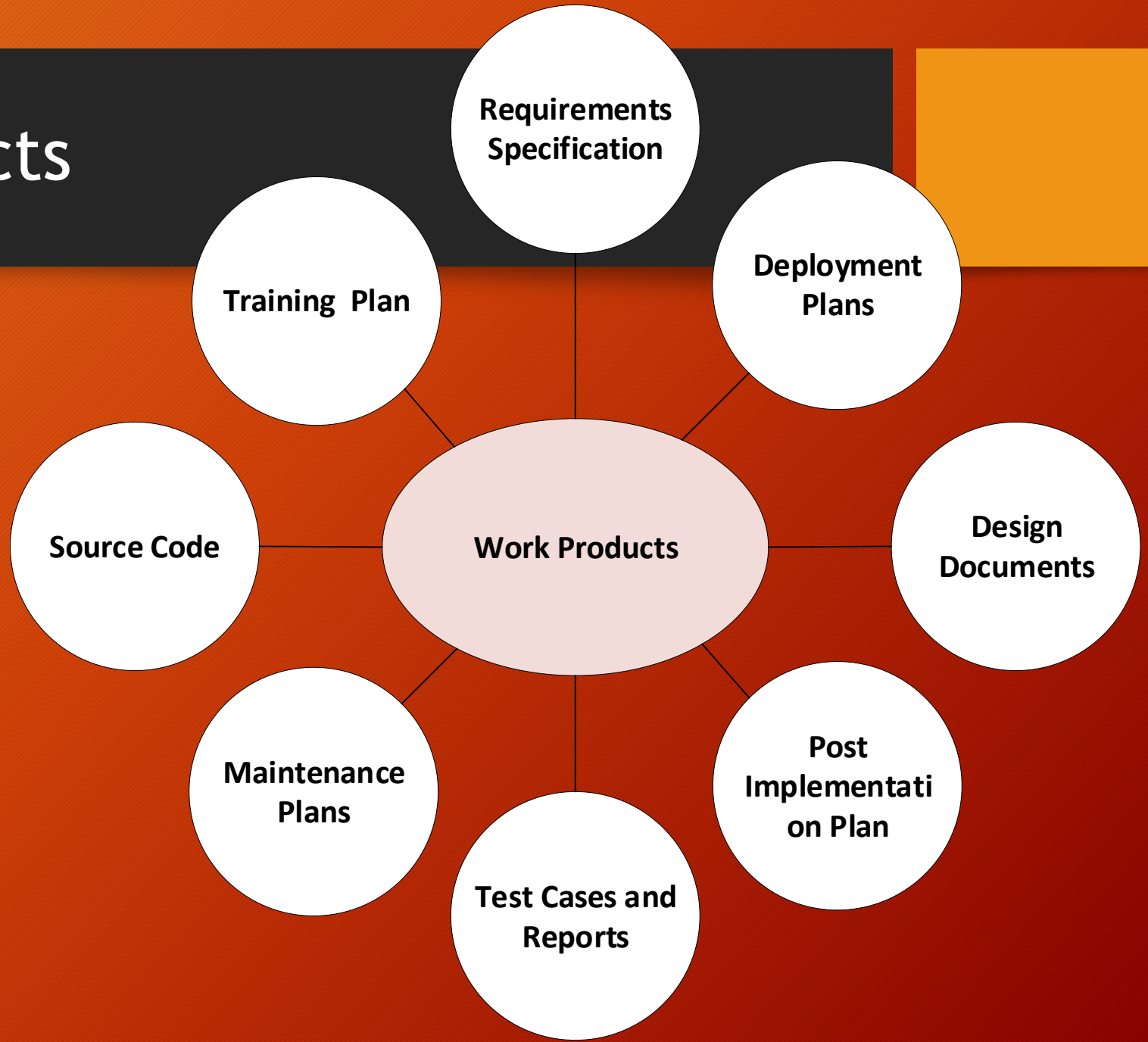
Systems and Models

- Software systems are often represented using various models that help visualize, analyze, and design complex systems.
- **System**: A collection of components organized to achieve specific goals. Examples: **Inventory systems, e-commerce platforms**.
- **Models**:
  - **Structural Models**: Represent system components and their relationships (e.g., class diagrams).
  - **Behavioral Models**: Depict the dynamic behavior of the system (e.g., use-case and sequence diagrams).
  - **Architectural Models**: Show the system's high-level structure and interactions.

Software Work Products

Other Work Products

- Risk Management Plan
- User Documentation
- Version Control Logs



Activities, Tasks, and Resources

- Software projects are divided into activities and tasks, executed using available resources.
- **Activities:** High-level processes that guide software development.
  - Examples: Requirement gathering, system design, coding, testing, and maintenance.
- **Tasks:** Specific actions under each activity.
  - Example: Writing a function as part of the coding activity.
- **Resources**: Include human (team members), technical (tools, hardware, and software), and time (project schedule).

Notations, Methods, and Methodologies

- Software engineering employs various notations, methods, and methodologies to structure and document processes.
- **Notations**: Visual representations of systems or processes.
  - Examples:
    - Unified Modeling Language (UML): Class diagrams, sequence diagrams, etc.
    - Entity-Relationship Diagrams (ERD): For database design.
- **Methods**: Structured techniques used to execute specific tasks.
- Examples:
  - Object-Oriented Analysis and Design (OOAD).
  - Agile practices like Scrum sprints

Notations, Methods, and Methodologies

- **Methodologies**: Overarching frameworks guiding the entire development lifecycle.
- Examples:
  - Waterfall Model: Sequential phases.
  - Agile: Iterative and incremental.
  - DevOps: Combines development and operations for continuous delivery

Overview of Software Process Models

- A **Software Process Model** is a structured framework that defines the steps, activities, and tasks required to develop software effectively. These models help teams organize work, manage resources, and ensure quality throughout the software development lifecycle
- Traditional Models:
  - Waterfall, Iterative, Spiral
  - Agile Models

Criteria	Waterfall Model	Agile Model	Spiral Model	V-Model	Incremental Model	Prototype Model	DevOps Model
Description	Sequential model with distinct phases.	Iterative, incremental development with flexibility.	Iterative model emphasizing risk management.	Enhanced Waterfall with testing at each stage.	Delivers functionality in increments.	Develops a prototype to refine requirements.	Combines development and operations for continuous delivery.
Best Use Case	Well-defined, stable requirements.	Projects with evolving requirements and frequent changes.	High-risk, complex projects requiring frequent evaluation.	Projects requiring rigorous validation and quality control.	Projects requiring gradual functionality delivery.	Unclear or ambiguous requirements.	Projects needing continuous integration and frequent delivery.
Flexibility	Low	High	Moderate	Low	Moderate	High	High
Customer Involvement	Limited to early phases.	High; continuous feedback and interaction.	Moderate; involved during reviews.	Low; involved in the beginning.	Moderate; involved for each increment.	High; feedback on prototypes.	High; feedback during deployments.
Risk Management	Low	Moderate	High; focuses on risk mitigation.	Low	Moderate	Moderate	High
Delivery	At the end of all phases.	Frequent delivery through sprints.	Cyclical delivery in stages.	Final delivery after validation.	Delivered incrementally.	Delivered incrementally.	Continuous delivery of updates.

Criteria	Waterfall Model	Agile Model	Spiral Model	V-Model	Incremental Model	Prototype Model	DevOps Model
Time Efficiency	Slow due to sequential phases.	Faster due to iterative development.	Slower; cycles add time but reduce risk.	Moderate due to validation phases.	Faster for initial delivery but depends on increments.	Faster to clarify requirements.	Fast; focuses on automation and continuous updates.
Cost Efficiency	Predictable costs for stable projects.	Costs may increase with frequent changes.	High initial costs for risk management.	Predictable but rigid costs.	Moderate; incremental effort affects costs.	Moderate to high for building prototypes.	High for initial automation setup but reduces long-term costs.
Quality Control	Low; testing occurs late.	High; iterative testing improves quality.	High; continuous evaluation and testing.	High; emphasis on validation.	Moderate; quality improves over increments.	Moderate; quality depends on prototype feedback.	High; ensures high quality through CI/CD pipelines.
Scalability	Low; difficult to scale.	High; supports modular and evolving requirements.	High; accommodates changes and risks.	Low; rigid structure limits scaling.	High; can adapt to new increments.	Low; limited to the prototype scope.	High; supports scaling through automation and modular architecture.
Examples	Banking software, government projects.	Social media apps, startups.	Aerospace, defense systems.	Medical software, safety-critical systems.	E-commerce platforms, mobile apps.	Proof-of-concept projects, UI-heavy apps.	SaaS platforms, cloud services.

Brain Teaser

Scenario Analysis: "A startup wants to develop a new mobile application to enter a competitive market quickly. They have a rough idea of the essential features but expect to refine the app based on user feedback after launch. Which software development model would you recommend and why?"

- **Use the knowledge acquired in the lecture and provide practical justifications for your choices.**
- **Your solutions should be practical and based on real world practices.**

Answer

Recommended Model: Agile Reason:

For a startup looking to enter a competitive market quickly with a mobile app, the Agile model is ideal.

It supports rapid development and frequent iteration based on user feedback.

This model allows the startup to continuously refine and improve the app in response to real user experiences, which is crucial when initial requirements are not fully defined and may evolve.

Agile facilitates quick releases and adaptive planning, which are essential for startups in dynamic market environments.

Brain Teaser

Consider a project for a banking system that requires high reliability and security, and the requirements are well-defined but complex. Which process model would you suggest, and what are the key factors influencing your choice?"

- **Use the knowledge acquired in the lecture and provide practical justifications for your choices.**
- **Your solutions should be practical and based on real world practices.**

Answer

Waterfall Model

- For a banking system where reliability and security are paramount, and the requirements are complex but well-defined, the Waterfall model is suitable.
- This model's sequential nature ensures that thorough documentation and planning occur before development starts, reducing the likelihood of errors in critical systems.
- Key factors influencing this choice include the need for stringent phase completion checks, regulatory compliance, and the high cost of mistakes, which necessitates a more predictable and structured approach.

Brain Teaser

Imagine you are a project manager in a software company tasked with developing a large e-commerce platform. The client is unsure about some features and expects to make changes based on the market trends. Which development model would you choose and how would you justify this to your team?

- **Use the knowledge acquired in the lecture and provide practical justifications for your choices.**
- **Your solutions should be practical and based on real world practices.**

Answer

Agile Justification

- Given the uncertainty about some features and the need for flexibility to adapt to market trends, Agile is the preferred model.
- This approach would allow your team to work in sprints, providing opportunities to adjust the development path based on feedback and new requirements.
- This flexibility is crucial for an e-commerce platform, where user experience and market needs can change rapidly.
- You would justify this choice to your team by emphasizing the importance of adaptability and continuous improvement in meeting client and market needs effectively.

What is Requirements Engineering?

- Process of defining, documenting, and maintaining software requirements.
- Ensures alignment between user needs and the final software product.
- Involves gathering, analyzing, specifying, and validating requirements.

Why is Requirements Engineering Important?

- Ensures clear understanding of what the system must do.
- Reduces the risk of costly rework during development.
- Aligns stakeholder expectations with the final product.
- Supports better project planning, estimation, and risk management.

Types of Requirements

- **Functional Requirements**: Define what the system should do.
- **Non-functional Requirements**: Define system attributes like performance, security, etc.
- **Stakeholder Requirements**: Capture needs and expectations from users and stakeholders.
- **System Requirements**: Translate stakeholder needs into technical specifications.

What are Functional Requirements?

- Define system functionality, behavior, and operations.
- Describe inputs, outputs, data processing, and user interactions.
- Examples:
- User login authentication.
- Data validation and error handling.
- Payment processing in an e-commerce application.

Characteristics of Functional Requirements

- Must be clear, concise, and unambiguous.
- Should be verifiable through testing or inspection.
- Should be complete, covering all aspects of required functionality.
- Should be feasible to implement within technical and financial constraints.

What are Non-functional Requirements?

- Define system qualities, constraints, and performance criteria.
- Focus on how the system should operate rather than what it should do.
- Examples:
- - Performance: System should respond within 2 seconds.
- - Security: Multi-factor authentication for all users.
- - Usability: Intuitive and easy-to-use user interface.

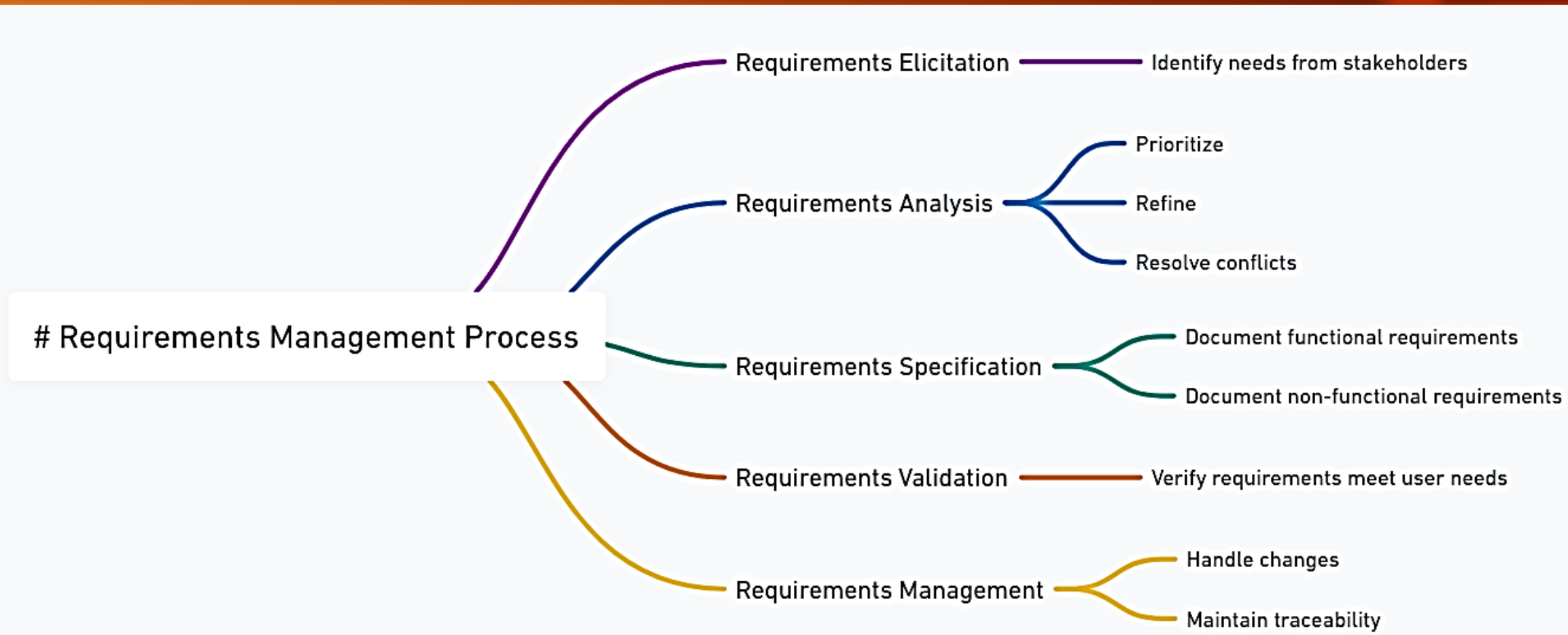
Categories of Non-functional Requirements

- Performance: Speed, response time, and throughput.
- Security: Data protection, authentication, and authorization.
- Usability: User-friendliness, ease of use, and accessibility.
- Reliability: System availability, fault tolerance, and recovery.
- Maintainability: Ease of updates, patches, and upgrades.

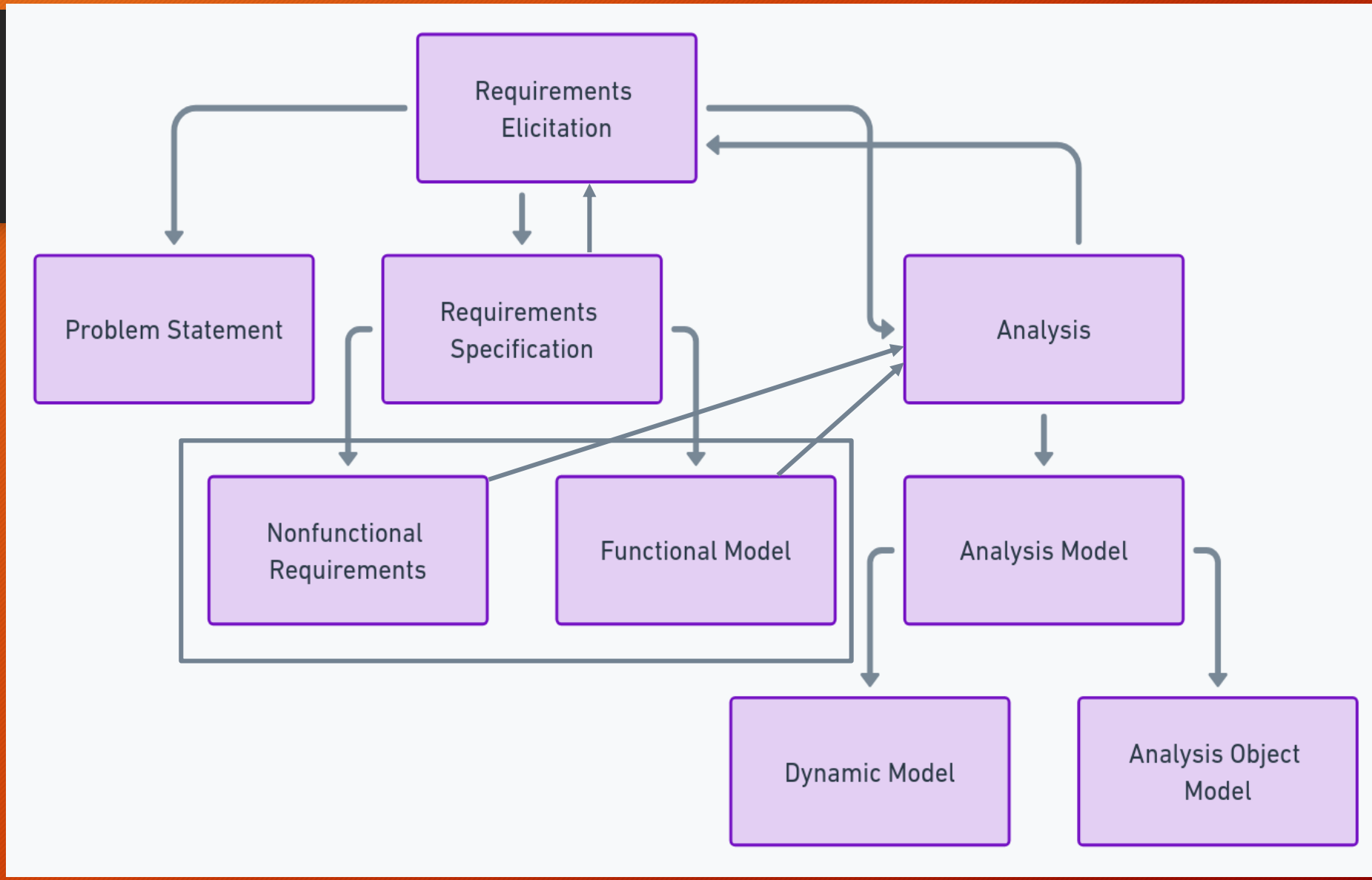
Importance of Requirements Engineering

- Provides a shared understanding of system goals and expectations.
- Reduces development time and cost by avoiding rework.
- Ensures user satisfaction by meeting their needs and expectations.
- Supports better decision-making and project scope management.

Requirements Engineering Process



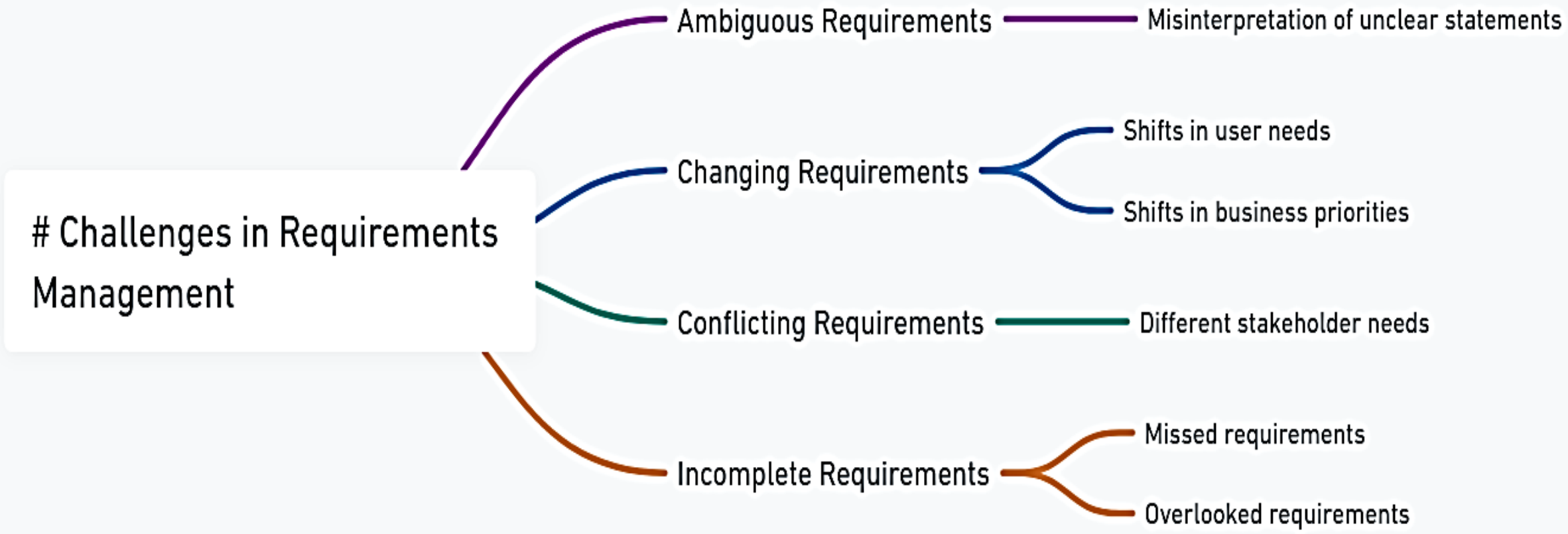
Requirements Elicitation



Techniques for Requirements Elicitation

- Interviews: One-on-one discussions with stakeholders.
- Surveys/Questionnaires: Collect requirements from large groups.
- Observation: Watch users interact with existing systems.
- Workshops: Collaborative sessions to identify and prioritize needs.
- Prototyping: Build prototypes to gather feedback from users.

Challenges in Requirements Engineering



Best Practices for Effective Requirements Engineering

- Involve stakeholders early and frequently.
- Document and validate all requirements.
- Prioritize requirements based on business value.
- Use clear, unambiguous language in requirements documents.
- Continuously review and update requirements throughout the project.

Final Thoughts on Requirements Engineering

- Requirements Engineering is critical for project success.
- Clearly defined requirements reduce rework and increase satisfaction.
- Both functional and non-functional requirements play vital roles.
- Effective Requirements Engineering enables better planning, development, and user experience.

Reflection from Vasa Tragedy

- In what ways did evolving and undocumented specifications affect the Vasa's design and construction, and how can software engineers ensure that changing requirements do not compromise project stability and success?

Reflection fro Vasa Tragedy

- How did the lack of clear communication and coordination among the Vasa's designers, builders, and naval officers contribute to its failure, and what lessons can be applied to modern software project management to prevent similar issues?
- Given the Vasa project's challenges with changing requirements and design flaws, which software development process model (e.g., Waterfall, Agile, Spiral) would be most appropriate to handle similar situations, and why?

Thank You

Any queries??